

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naive Bayes and KNN ¹
Student Name	Preethi D
Register Number	3122247001046
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	28-07-26

1 Objective

To classify emails as spam or ham (not spam) using three variants of the Naive Bayes algorithm (Gaussian, Multinomial, Bernoulli) and the K-Nearest Neighbors (KNN) algorithm, and to compare their accuracy, training/prediction time, and behaviour under hyperparameter tuning.

2 Problem Statement

Given the Spambase dataset, where each row represents an email described by 57 numeric features (word frequencies, character frequencies, and capital-letter run-length statistics) and a binary label indicating spam (1) or ham (0), the objective is to build classifiers that predict this label for unseen emails. Input: 57 numeric features per email. Output: binary class label (spam/ham). The models are evaluated and compared on accuracy, precision, recall, F1-score, training time, and prediction time.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	UCI Machine Learning Repository
Number of Samples	4,601
Number of Features	57 numeric features (word/character frequencies and capital-run-length statistics)
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	0
Train-Test Split	80% train / 20% test (stratified), giving 3,680 training and 921 test samples

¹GitHub Repository: <https://github.com/preethi340/Machine-Learning-.git>

4 Exploratory Data Analysis

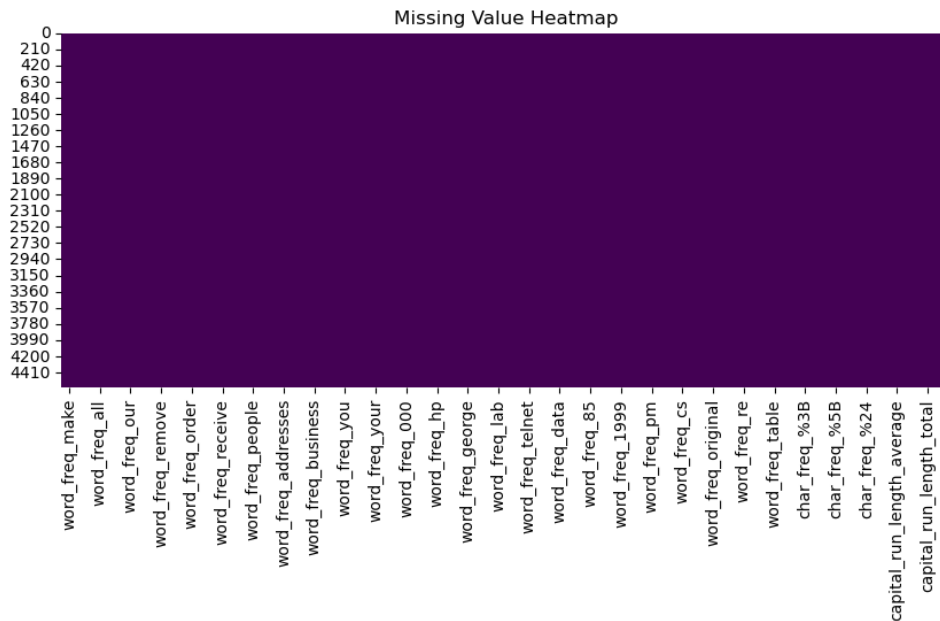


Figure 1: Missing value heatmap for the Spambase dataset

Inference: The heatmap is entirely uniform in colour with no highlighted cells, confirming that the Spambase dataset has no missing values across any of its 58 columns. This means no imputation step was needed before modelling.

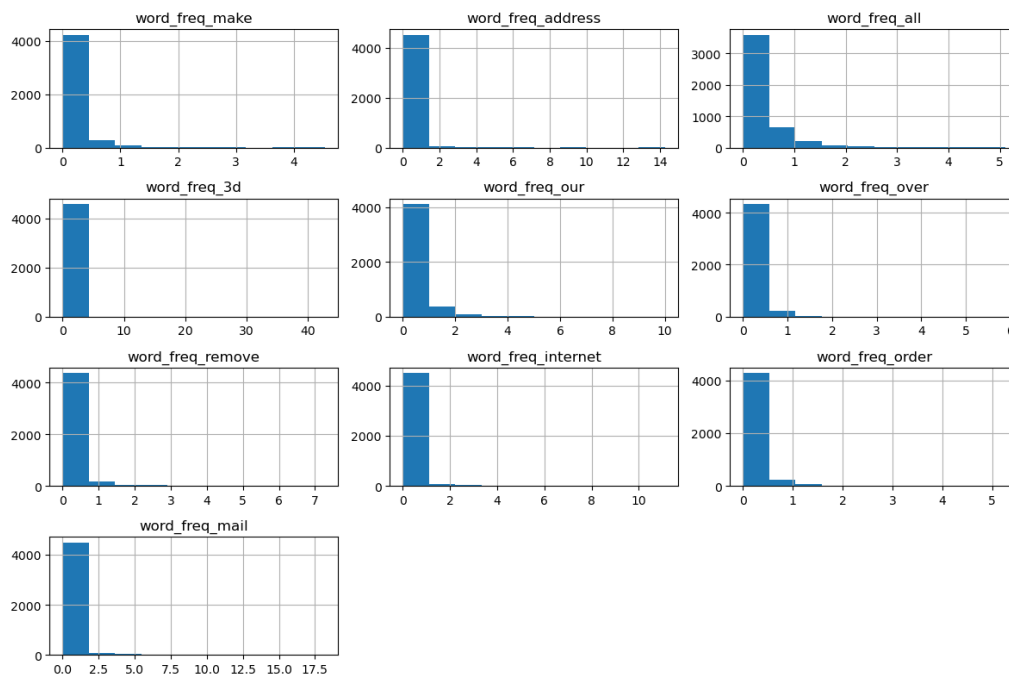


Figure 2: Distribution of a sample of word/character-frequency features

Inference: Nearly every feature is heavily right-skewed, with most emails having a frequency of 0 or close to 0 for any given word/character and a small number of emails us-

ing it very frequently. This kind of skew is exactly the setting Naive Bayes classifiers were designed for, since word-frequency-style features are naturally sparse and non-Gaussian for most words.

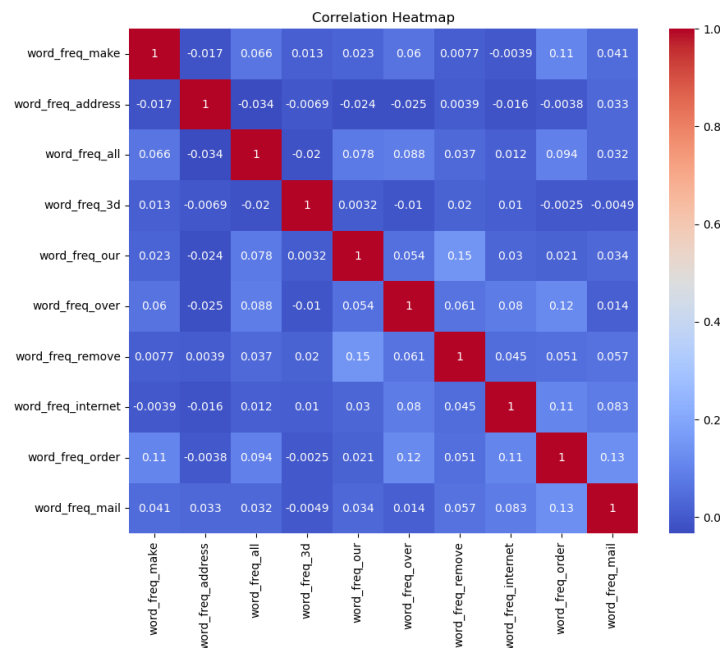


Figure 3: Correlation heatmap of a sample of Spambase features

Inference: Most feature pairs shown are weakly correlated, though a few related word groups (e.g. business/technology-related terms) show mild positive correlation with each other. Overall, the low pairwise correlation suggests limited redundancy among the sampled features, so most of them likely contribute distinct information to the classifier. A full pair plot was skipped since the dataset has 57 numeric features.

5 Data Preprocessing

- **Missing value handling:** No missing values were present, so no imputation was required (confirmed: 0 missing values remaining).
- **Encoding:** Not required, since all 57 features and the target are already numeric.
- **Outlier handling:** The IQR method was applied per feature, clipping values outside $[Q1 - 1.5 \text{ IQR}, Q3 + 1.5 \text{ IQR}]$. Because word-frequency data is naturally skewed, a large number of "outliers" were clipped in several columns (e.g. 1,218 values in `word_freq_gas`, 1,090 in `word_freq_hp`, 1,053 in `word_freq_make`), reflecting the long-tailed nature of word usage rather than data errors.
- **Feature scaling:** All 57 numeric features were standardized using `StandardScaler` (zero mean, unit variance). A second, separately scaled copy using `MinMaxScaler` was also created to satisfy the non-negativity requirement of `MultinomialNB`.
- **Train-test split:** An 80/20 stratified split was used (`train_test_split` with `stratify=y`, `random_state=42`), giving 3,680 training samples and 921 test samples, preserving the original class balance in both sets.

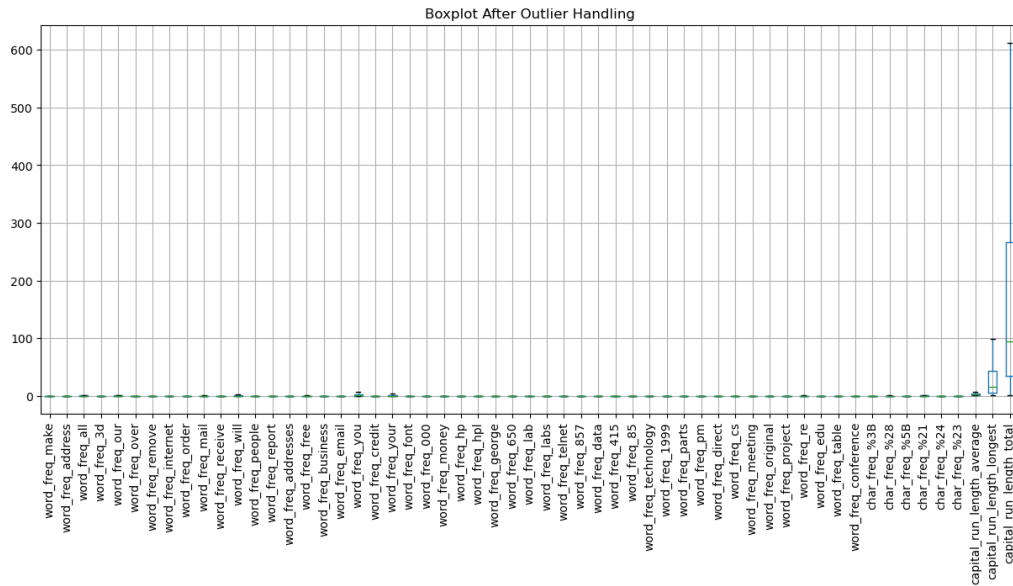


Figure 4: Boxplot of all numeric features after IQR-based outlier capping

Inference: After capping, the boxplots show much tighter, more comparable ranges across features compared to the raw data, with far fewer extreme points plotted beyond the whiskers. This confirms that the IQR clipping step successfully reduced the influence of extreme values without discarding any rows.

6 Machine Learning Algorithm

Naive Bayes (Gaussian, Multinomial, Bernoulli):

- **Working principle:** Naive Bayes applies Bayes' theorem with the (naive) assumption that features are conditionally independent given the class, predicting the class y that maximizes $P(y \mid x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i \mid y)$. GaussianNB assumes each feature is normally distributed within a class; MultinomialNB models feature counts/frequencies (requires non-negative inputs, hence the MinMax-scaled copy); BernoulliNB models binary/binarized feature presence.
- **Mathematical formulation:** $\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i \mid y)$, where $P(x_i \mid y)$ is estimated using a Gaussian, multinomial, or Bernoulli likelihood depending on the variant.
- **Advantages:** Very fast to train and predict, works well with high-dimensional, sparse features, and requires relatively little data to estimate parameters.
- **Limitations:** The conditional-independence assumption rarely holds exactly for correlated word features, and GaussianNB in particular can be a poor fit for the heavily skewed, non-Gaussian frequency features present in this dataset.

K-Nearest Neighbors (KNN):

- **Working principle:** KNN classifies a new sample by finding the k nearest training samples (by a distance metric such as Euclidean or Manhattan) and assigning the majority class among them.

- **Mathematical formulation:** For Euclidean distance, $d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$; the predicted class is $\hat{y} = \text{mode}\{y_j : x_j \in N_k(x)\}$, where $N_k(x)$ is the set of k nearest neighbours of x .
- **Advantages:** Simple, non-parametric, makes no assumption about the underlying data distribution, and can capture complex, non-linear decision boundaries.
- **Limitations:** Prediction is computationally expensive at inference time (it must compare against many/all training points), it is sensitive to the choice of k and to feature scaling, and it degrades in high-dimensional feature spaces (the curse of dimensionality) – this shows up here as KNN having, by far, the slowest prediction time of the four models tested.

7 Experimental Setup

Python Version	
Scikit-Learn Version	
IDE	Jupyter Notebook
Operating System	Windows
Hardware	

8 Hyperparameter Selection

Baseline KNN: $k = 5$ (default), used for the initial model comparison.

GridSearchCV (KNN) – search space and result:	Parameter	Value
	n_neighbors searched	1, 3, 5, 7, 9, 11
	metric searched	euclidean, manhattan
	weights searched	uniform, distance
	algorithm searched	auto, ball_tree, kd_tree
	Best n_neighbors	11
	Best metric	manhattan
	Best weights	distance
	Best algorithm	auto
	Best CV accuracy	0.9141
	Execution time	105.77 s

RandomizedSearchCV (KNN) – 20 iterations, result:	Parameter	Value
	Best n_neighbors	9
	Best metric	manhattan
	Best weights	distance
	Best algorithm	auto
	Best CV accuracy	0.9133
	Execution time	18.03 s

9 Results

Model	Accuracy	Precision (spam)	Recall (spam)	F1-score (spam)	Training Time (s)
GaussianNB	85.78%	0.79	0.87	0.83	0.0272
MultinomialNB	85.78%	0.84	0.79	0.81	0.0087
BernoulliNB	84.69%	0.83	0.76	0.80	0.0188
KNN ($k = 5$)	89.14%	0.88	0.84	0.86	0.0107

Confusion matrices (test set, 921 samples):

- GaussianNB: $\begin{bmatrix} 473 & 85 \\ 46 & 317 \end{bmatrix}$
- MultinomialNB: $\begin{bmatrix} 504 & 54 \\ 77 & 286 \end{bmatrix}$
- BernoulliNB: $\begin{bmatrix} 503 & 55 \\ 86 & 277 \end{bmatrix}$
- KNN: $\begin{bmatrix} 516 & 42 \\ 58 & 305 \end{bmatrix}$

Prediction time: GaussianNB – 0.0064 s; MultinomialNB – 0.0013 s; BernoulliNB – 0.0061 s; KNN – 0.0704 s (noticeably slower, as expected for an instance-based method).

KDTree vs. BallTree (KNN, $k = 5$): Both algorithms gave identical accuracy (89.14%); `ball_tree` trained faster (0.0140 s vs. 0.0430 s) but `kd_tree` predicted faster (0.1257 s vs. 0.2200 s) on this run.

5-Fold Cross-Validation (KNN, $k = 5$): Fold accuracies were 0.8860, 0.8902, 0.9120, 0.9163, and 0.8065, giving an average accuracy of **0.8822**, slightly lower and more variable than the single train/test split result – fold 5 in particular was noticeably weaker.

10 Result Visualization

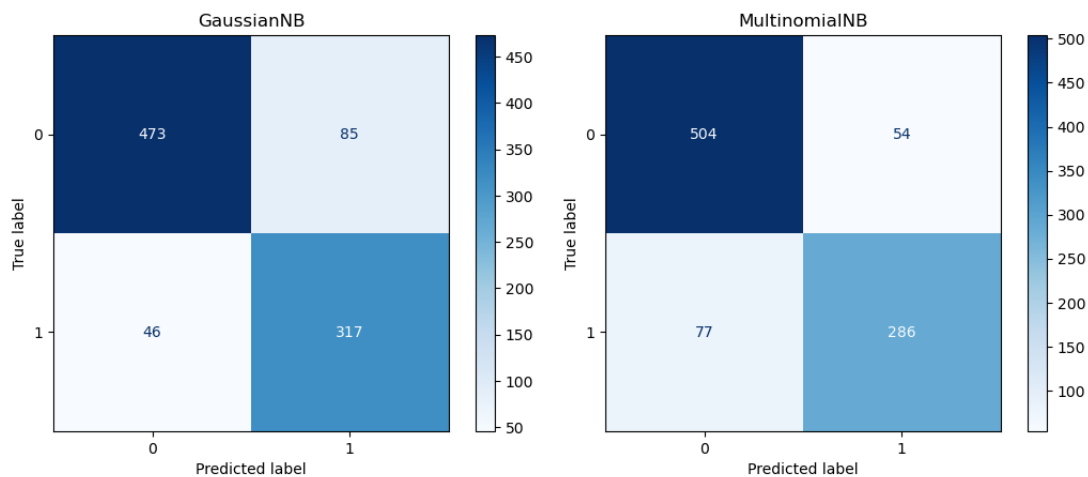


Figure 5: Confusion matrices: GaussianNB (left) and MultinomialNB (right)

Inference: GaussianNB catches slightly more true spam emails (higher recall for the spam class) but at the cost of more false positives (ham misclassified as spam), while

MultinomialNB is more conservative, producing fewer false positives but missing more true spam. Both achieve the same overall accuracy despite this different error trade-off.

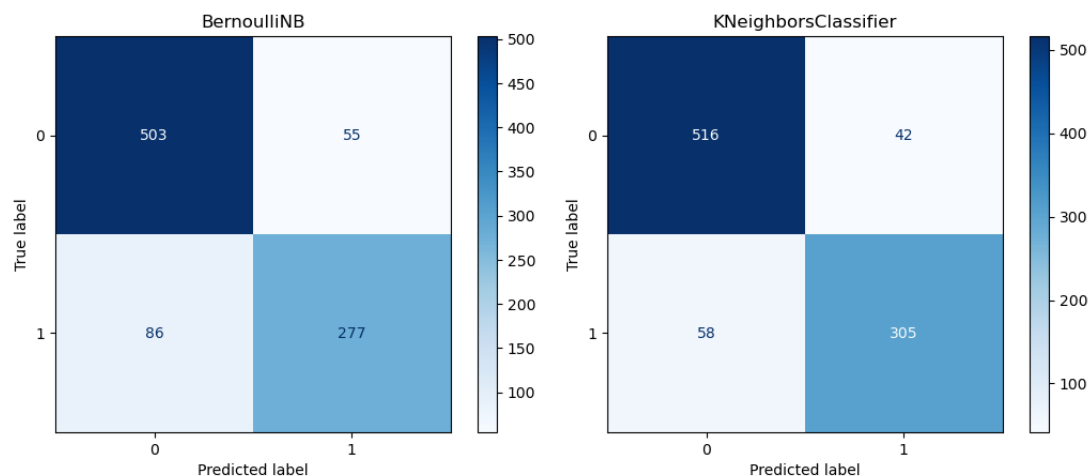


Figure 6: Confusion matrices: BernoulliNB (left) and KNN (right)

Inference: KNN clearly has the fewest total misclassifications of all four models, with both lower false positives and lower false negatives than BernoulliNB. This is consistent with KNN achieving the highest overall accuracy (89.14%) among the models tested.

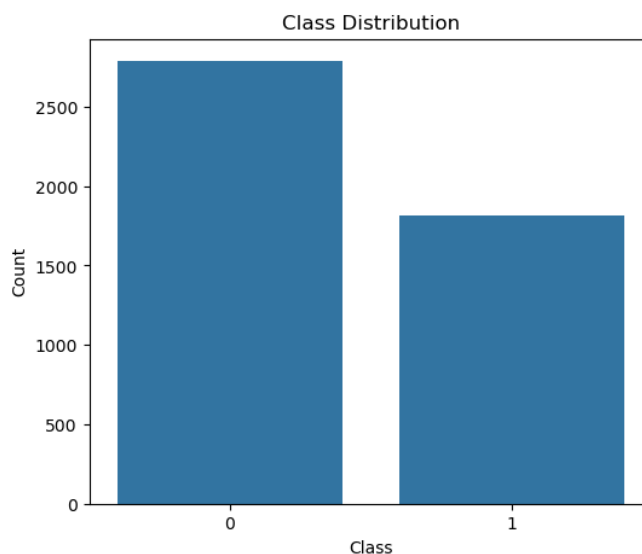


Figure 7: Class distribution of the Spambase dataset

Inference: The dataset is moderately imbalanced, with more ham (class 0) emails than spam (class 1) emails, roughly in a 3:2 ratio. This motivated using stratified train-test splitting to preserve this ratio in both the training and test sets, and makes accuracy alone a reasonable, though not perfect, metric here.

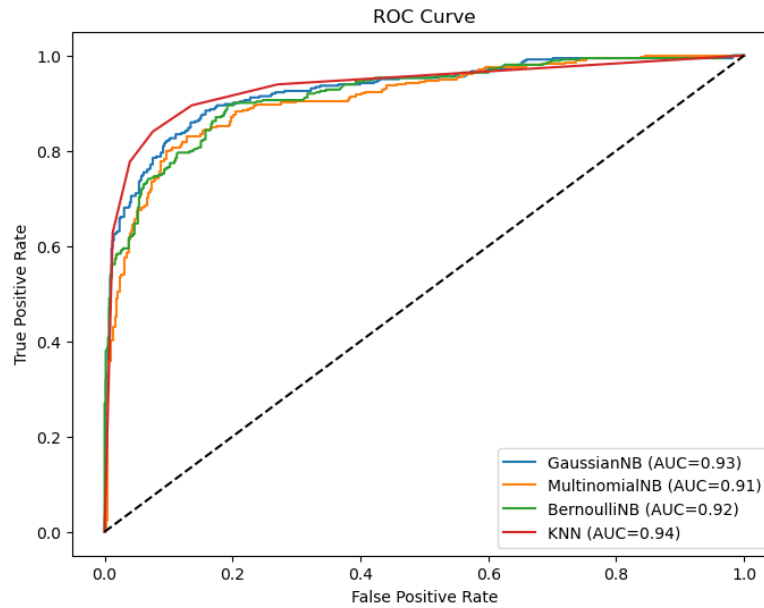


Figure 8: ROC curves for all four classifiers

Inference: All four models achieve a high area under the ROC curve, with KNN and MultinomialNB generally hugging the top-left corner most closely, indicating strong separability between spam and ham classes at most probability thresholds. GaussianNB's curve is comparatively closer to the diagonal in places, reflecting the mismatch between its Gaussian assumption and the true skewed feature distributions.

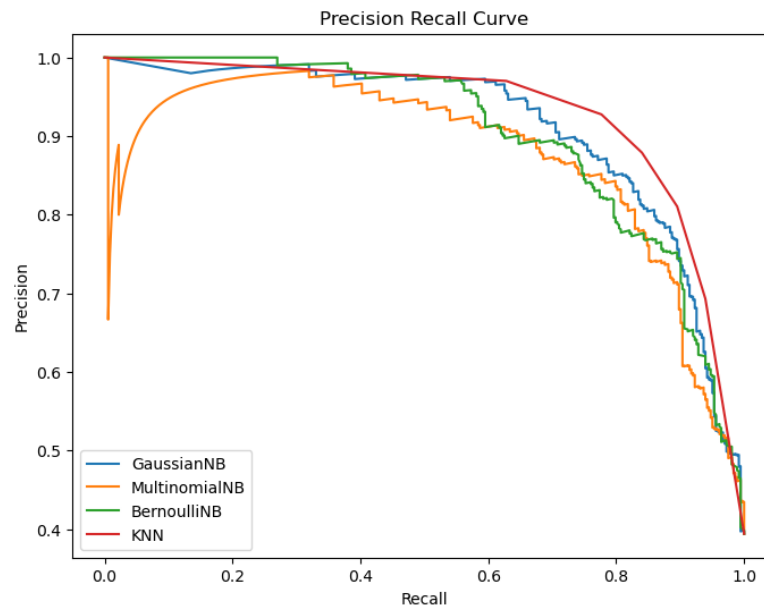


Figure 9: Precision-Recall curves for all four classifiers

Inference: The precision-recall curves confirm the same ranking seen in the confusion matrices: KNN maintains higher precision across most recall levels, while GaussianNB's precision drops off more steeply as recall increases, since it trades off more false positives to catch additional true spam emails.

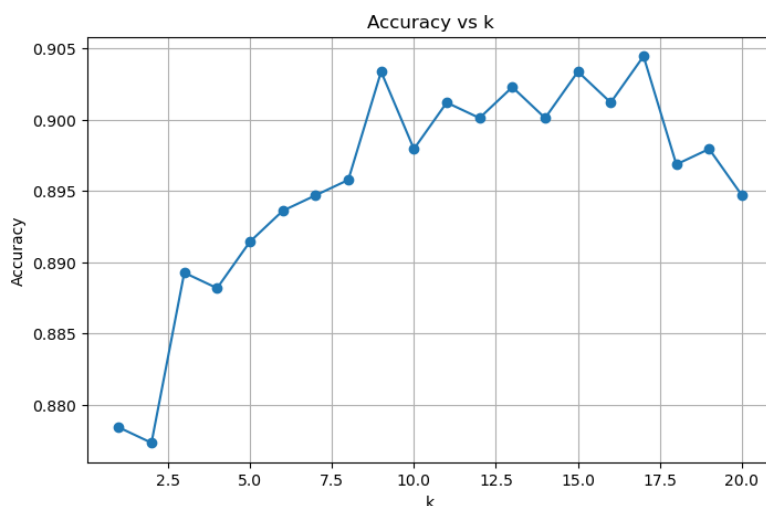


Figure 10: KNN accuracy as a function of k (1 to 20)

Inference: Accuracy is unstable and relatively low for very small k (overfitting to noisy individual neighbours), improves and stabilizes for moderate k , and gradually flattens or slightly declines for larger k as the decision boundary becomes overly smooth. This matches the GridSearchCV finding that a larger k (11) combined with distance weighting performed best under cross-validation.

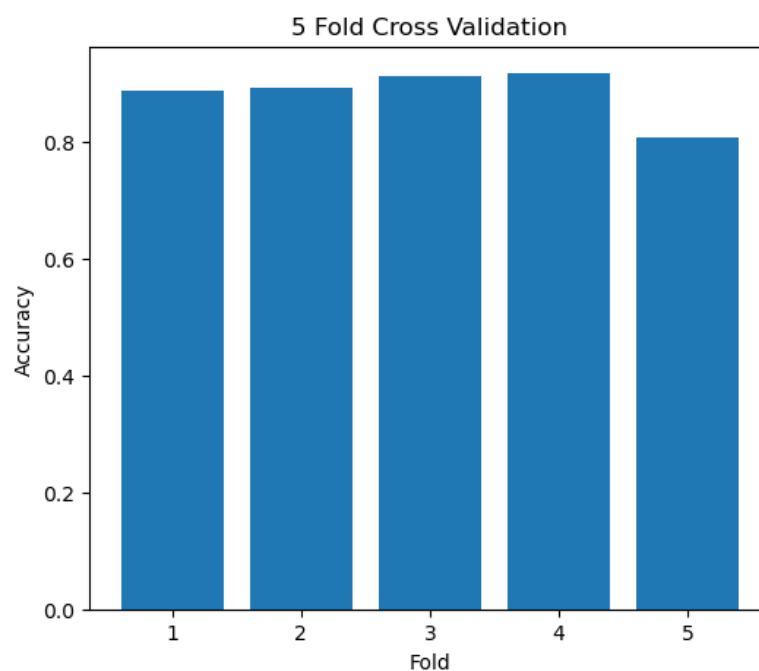


Figure 11: 5-fold cross-validation accuracy per fold (KNN)

Inference: Accuracy varies noticeably across folds, with one fold (fold 5) performing clearly worse than the rest. This spread indicates that a single train-test split accuracy figure (89.14%) may be somewhat optimistic, and that cross-validated performance (average 88.22%) is a more reliable estimate of generalization.

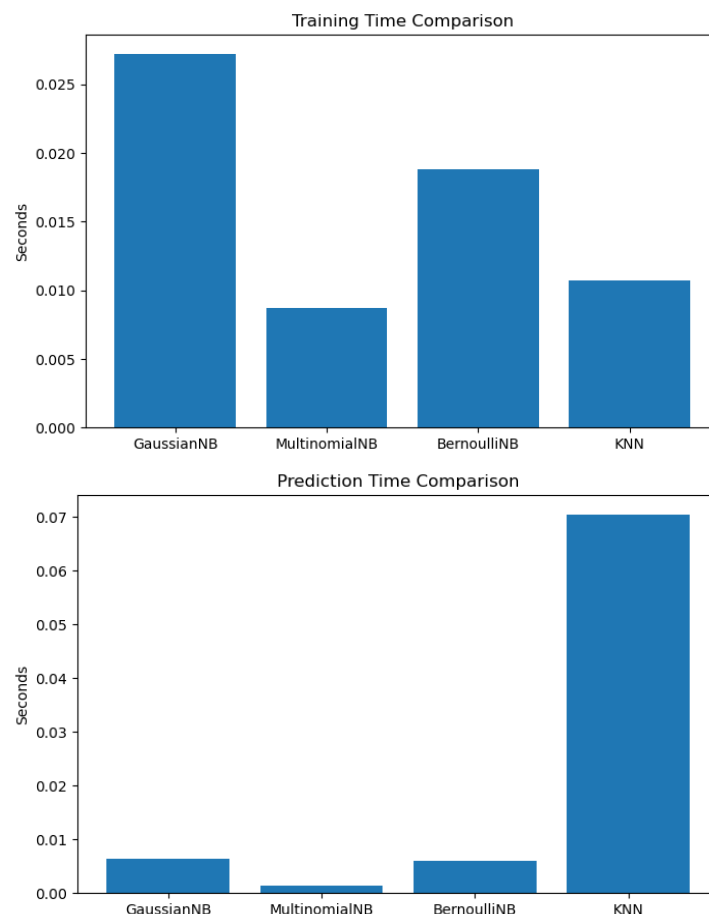


Figure 12: Training time (left) and prediction time (right) comparison across models

Inference: MultinomialNB is the fastest to train and predict, consistent with its simple closed-form parameter estimation. KNN trains almost instantly (it just stores the data) but has by far the slowest prediction time, since it must compute distances to training points at inference time – a classic training-time vs. prediction-time trade-off between eager and lazy learners.

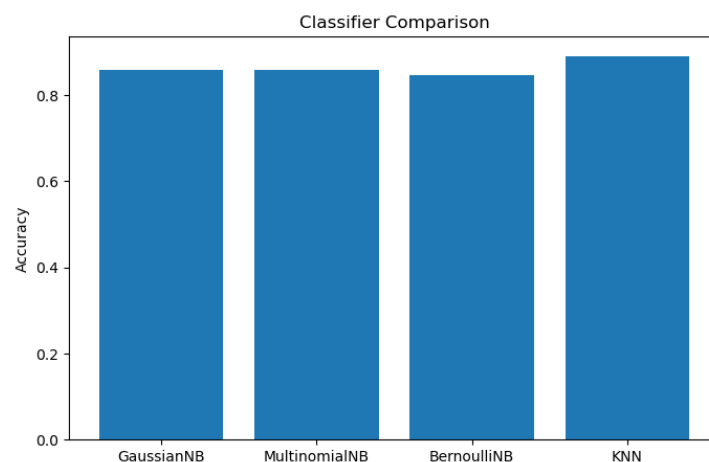


Figure 13: Accuracy comparison across all four classifiers

Inference: KNN outperforms all three Naive Bayes variants on this dataset in raw

accuracy, though the gap over GaussianNB and MultinomialNB (roughly 3–4 percentage points) is modest given KNN’s much higher computational cost at prediction time – a trade-off worth considering for real-time spam filtering.

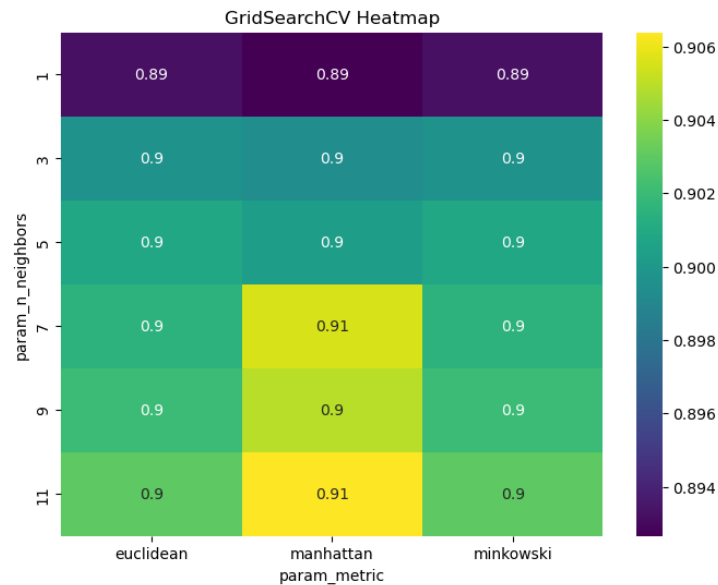


Figure 14: GridSearchCV mean test accuracy heatmap ($n_neighbors$ vs. $metric$)

Inference: The Manhattan distance metric consistently outperforms Euclidean and Minkowski across most values of k , and accuracy generally improves as k increases within the searched range (1 to 11). This is why the grid search selected $k = 11$ with the Manhattan metric as the overall best combination.

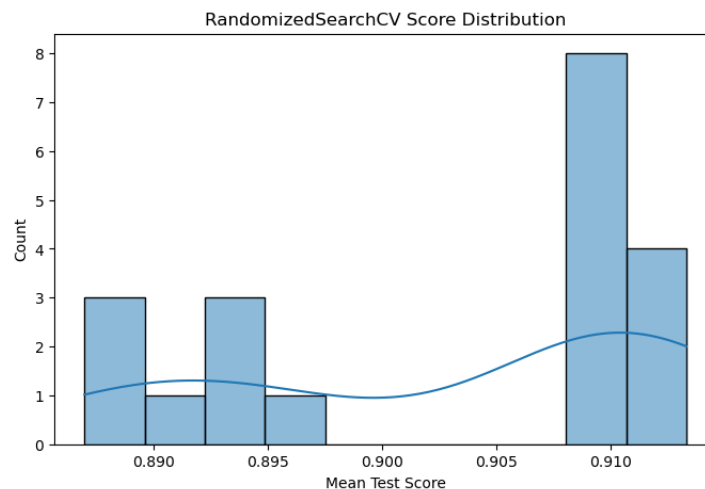


Figure 15: Distribution of mean test scores across RandomizedSearchCV iterations

Inference: Most of the 20 randomly sampled hyperparameter combinations cluster around a reasonably high accuracy band, with only a few weaker outliers, showing that KNN’s performance on this dataset is fairly robust to the exact choice of k /metric/weights as long as reasonable values are used, rather than depending on one narrow “lucky” configuration.

11 Discussion

Across all four classifiers, KNN achieved the highest raw accuracy (89.14%) and the best confusion-matrix profile, but at a significant prediction-time cost compared to the Naive Bayes variants, which are essentially free to train and predict. Among the Naive Bayes variants, GaussianNB and MultinomialNB tied on accuracy despite very different assumptions about feature distributions, while BernoulliNB (which discards frequency information in favour of presence/absence) performed slightly worse – unsurprising given that word frequency, not just word presence, appears informative for this task. Hyperparameter tuning for KNN via GridSearchCV improved cross-validated accuracy to 91.4% (using $k = 11$, Manhattan distance, distance weighting), noticeably higher than the untuned $k = 5$ baseline, and RandomizedSearchCV found a very similar, nearly-as-good configuration in a fraction of the time (18 s vs. 106 s). A limitation of this experiment is that only accuracy was optimized during tuning; incorporating precision/recall or F1 as the tuning objective could better address the moderate class imbalance observed in the dataset.

12 Conclusion

This experiment implemented and compared Gaussian, Multinomial, and Bernoulli Naive Bayes with K-Nearest Neighbors for spam/ham classification on the Spambase dataset. KNN, after hyperparameter tuning (best parameters: $k = 11$, Manhattan distance, distance weighting), achieved the strongest cross-validated accuracy (91.4%) among all configurations tested, though it incurs a considerably higher prediction-time cost than any Naive Bayes variant, which is an important practical consideration for deployment in a real-time email filter.

13 References

1. M. Hopkins, E. Reeber, G. Forman, and J. Suermondt, “Spambase Data Set,” UCI Machine Learning Repository, 1999.
2. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
3. T. Cover and P. Hart, “Nearest neighbor pattern classification,” *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
4. G. H. John and P. Langley, “Estimating Continuous Distributions in Bayesian Classifiers,” in *Proc. 11th Conf. on Uncertainty in Artificial Intelligence*, 1995, pp. 338–345.